

Intro to Deep Learning

Lecture 10: Object Detection and YOLO

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. How is object detection different from classification?
2. What does a detector output?
3. What are common bounding box formats?
4. What is IoU?
5. Why do we need confidence thresholds and NMS?
6. What is the high-level idea behind YOLO?

Lecture rhythm: boxes -> IoU -> thresholds/NMS -> YOLO outputs -> failure analysis

Recap From Lecture 09

Visual search returns similar examples.

Object detection answers a different question:

what objects are in this image
and where are they?

Instead of one label per image, we need:

multiple boxes + multiple class predictions

Part 1

Detection task and outputs

Classification vs Detection

Classification:

```
input image -> one class label
```

Object detection:

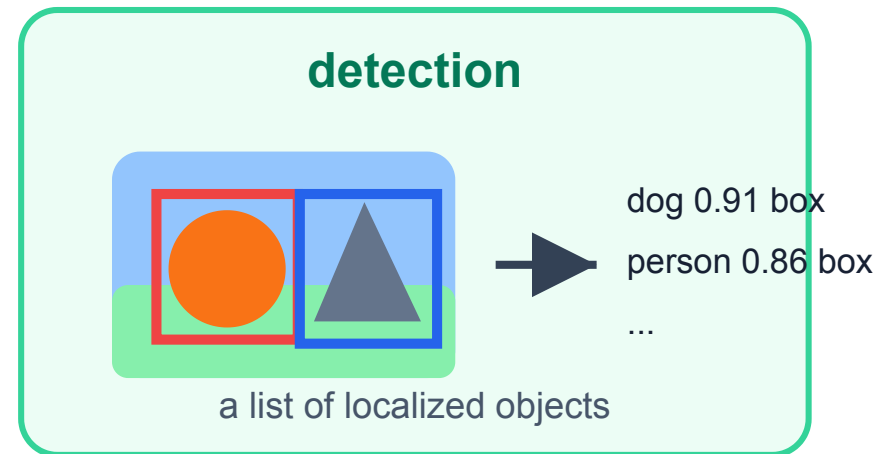
```
input image -> list of detected objects
```

Each detected object usually has:

```
class label  
bounding box  
confidence score
```

Classification vs Detection: Picture

Classification gives one label; detection gives objects and locations



Object Detection Output

A detector returns a variable-length list:

```
detection 1: class, confidence, box  
detection 2: class, confidence, box  
detection 3: class, confidence, box  
...
```

Different images can have different numbers of detections.

This makes detection more complex than classification.

Three Prediction Parts

For each candidate object:

Class:

what object is this?

Box:

where is it?

Confidence:

how sure is the detector?

Bounding Boxes

A bounding box is a rectangle around an object.

Common coordinate formats:

```
xyxy: x_min, y_min, x_max, y_max  
xywh: x_center, y_center, width, height
```

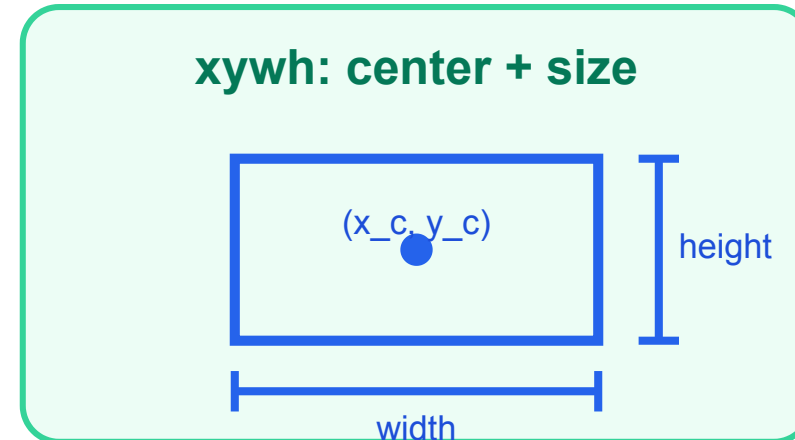
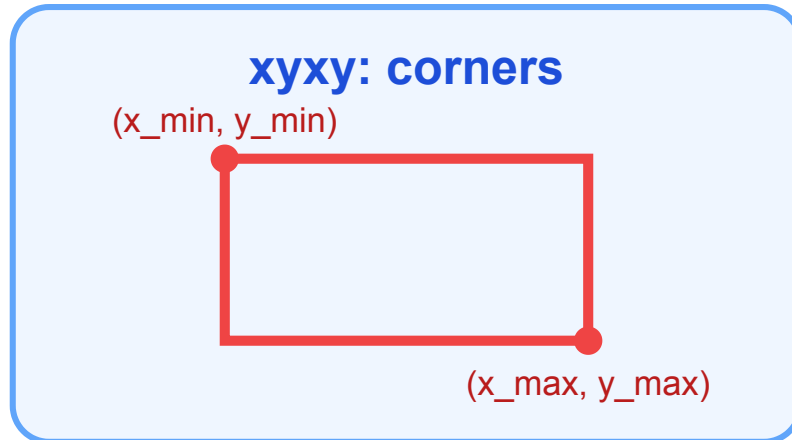
Some libraries use absolute pixels.

Others use normalized coordinates between 0 and 1.

Always check the format.

Box Formats: Picture

Always check the bounding-box coordinate format



Box Format Example

Suppose:

```
xyxy = [20, 30, 120, 90]
```

Then:

```
left    = 20  
top     = 30  
right   = 120  
bottom  = 90
```

Width and height:

```
width  = 120 - 20 = 100  
height = 90 - 30 = 60
```

Why Boxes Are Hard

Detection is harder than classification because the model must solve:

- how many objects are present
- which classes they belong to
- where each object is
- which overlapping predictions are duplicates
- which uncertain predictions to remove

This is why post-processing matters.

Part 2

Boxes, IoU, and overlap

Intersection Over Union

IoU measures overlap between two boxes.

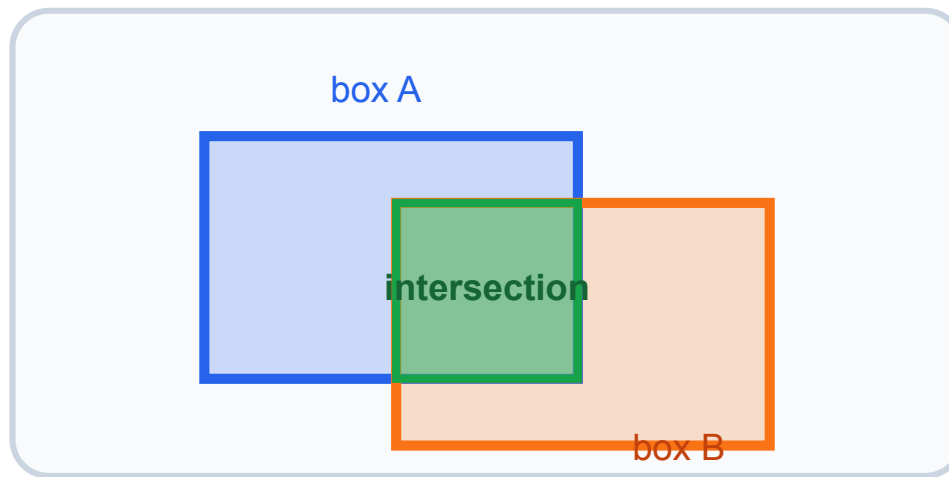
$$\text{IoU} = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)}$$

Interpretation:

IoU = 0: no overlap
IoU = 1: perfect overlap

IoU: Picture

IoU measures how much two boxes overlap



$$\text{IoU} = \frac{\text{intersection area}}{\text{union area}}$$

What IoU Is Used For

IoU appears in detection in several places:

- matching predictions to ground-truth boxes
- evaluating localization quality
- removing duplicate predictions
- setting NMS thresholds

IoU threshold often appears through evaluation matching and NMS behavior.

Part 3

Thresholds and NMS

Confidence Score

A confidence score estimates how likely a detection is useful.

Low confidence threshold:

more detections
more false positives

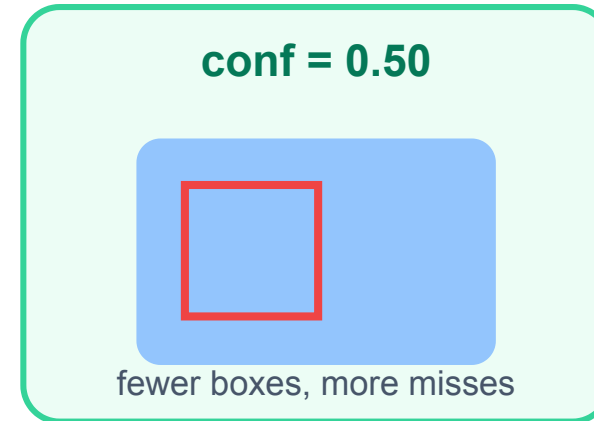
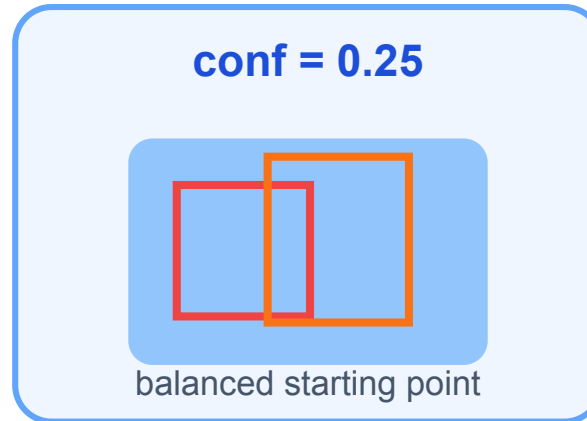
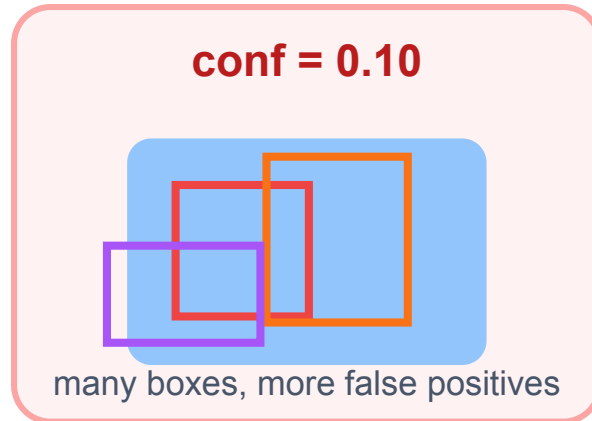
High confidence threshold:

fewer detections
more missed objects

The threshold controls a precision-recall tradeoff.

Confidence Threshold: Picture

Confidence threshold controls how many detections survive



Duplicate Detections

Detectors often produce several overlapping boxes for the same object.

Example:

```
box A: dog, confidence 0.91  
box B: dog, confidence 0.83  
box C: dog, confidence 0.64
```

If they describe the same object, we usually want to keep one.

This is why we use NMS.

Non-Maximum Suppression

NMS means:

```
keep the strongest box  
remove highly overlapping weaker boxes
```

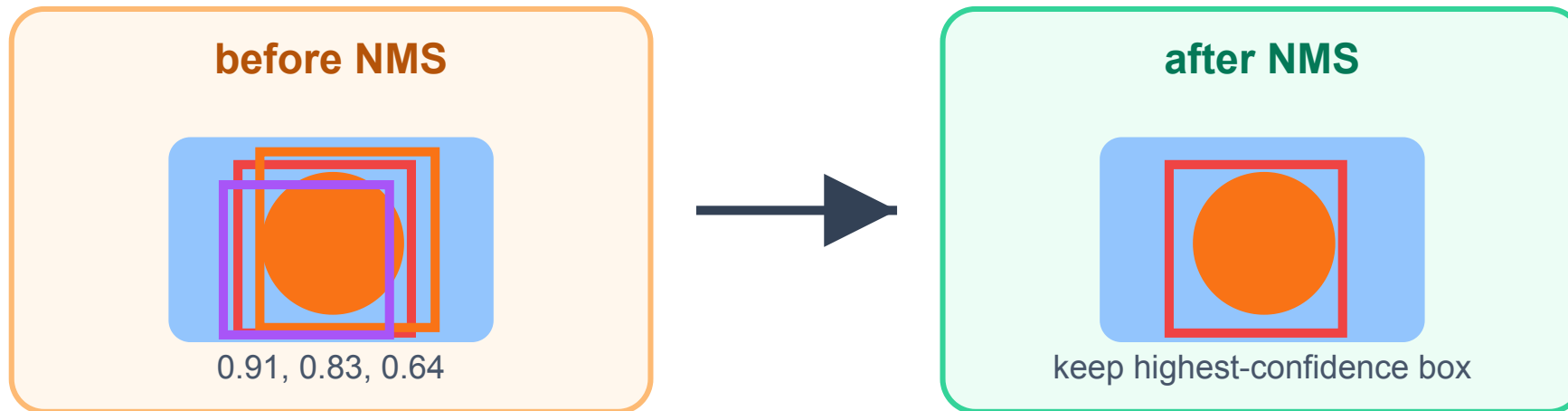
Simplified process:

1. sort boxes by confidence
2. keep the highest-confidence box
3. remove boxes with high IoU overlap
4. repeat

NMS is post-processing.

NMS: Picture

NMS removes duplicate boxes for the same object



IoU Threshold In NMS

NMS uses an IoU threshold to decide when boxes overlap too much.

Lower IoU threshold:

more aggressive removal
may remove nearby true objects

Higher IoU threshold:

less aggressive removal
may keep duplicate boxes

Threshold choice depends on the amount of overlap and crowding in the scene.

Mini-Check: Threshold Diagnosis

You run YOLO on a crowded image.

Symptoms:

- many duplicate boxes remain around the same objects
- increasing confidence removes some true objects
- lowering confidence makes the duplicates worse

Which threshold is more suspicious: confidence or NMS IoU?

What direction would you adjust first?

Answer

The more suspicious threshold is **NMS IoU**.

Many duplicate boxes mean NMS is not suppressing overlapping predictions strongly enough.

First try:

lower NMS IoU threshold

This makes NMS more aggressive.

Confidence threshold controls which candidates enter post-processing, but it does not directly solve duplicate boxes.

Part 4

YOLO inference pipeline

Two-Stage vs Single-Stage Detectors

Two-stage detectors:

```
first propose regions  
then classify/refine them
```

Single-stage detectors:

```
predict boxes and classes directly in one pass
```

YOLO is a single-stage detector.

This is why it is popular for fast practical detection.

YOLO Idea

YOLO stands for:

You Only Look Once

High-level idea:

```
process the image once  
predict many candidate detections  
filter them into final detections
```

Modern YOLO versions differ in details.

The practical mental model is stable.

What Does "Once" Mean?

YOLO does not crop many regions and classify them one by one.

It runs one neural network pass over the image.

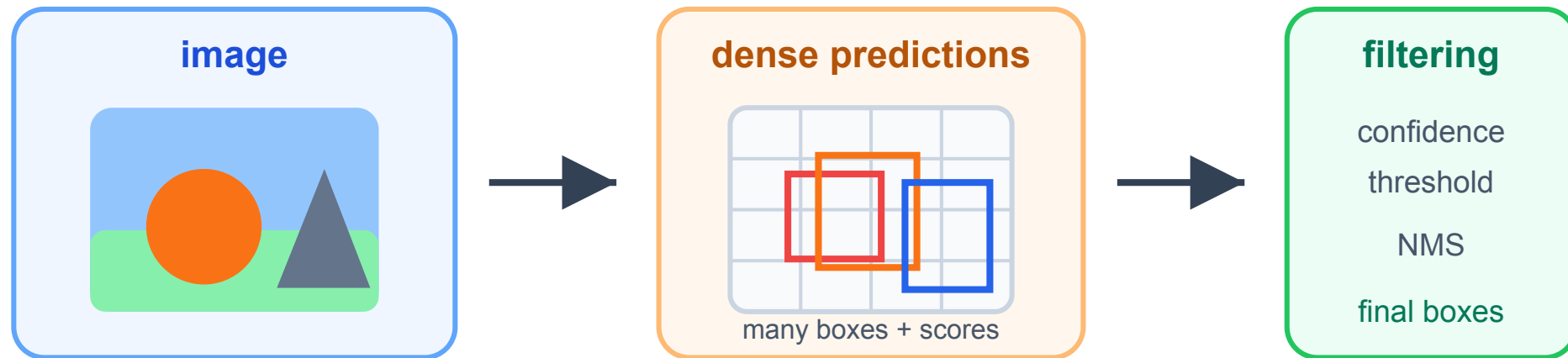
That pass produces many candidate detections:

- many possible boxes
- objectness or confidence scores
- class scores

Post-processing chooses the final boxes.

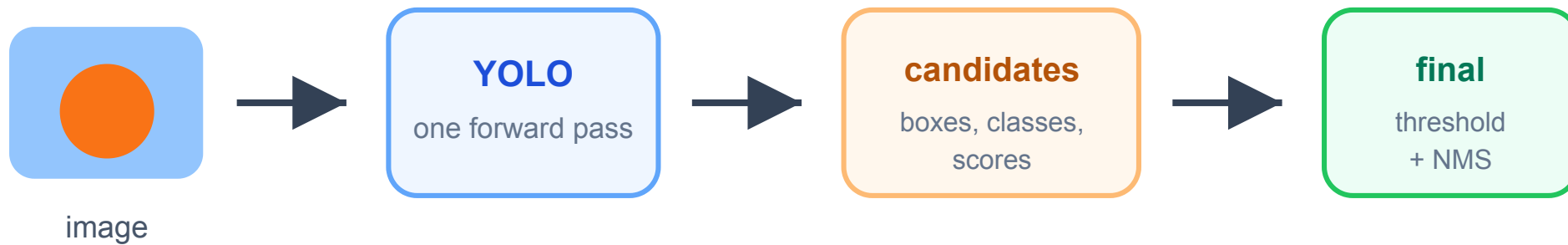
YOLO Dense Predictions

YOLO predicts many candidate boxes before choosing the final ones



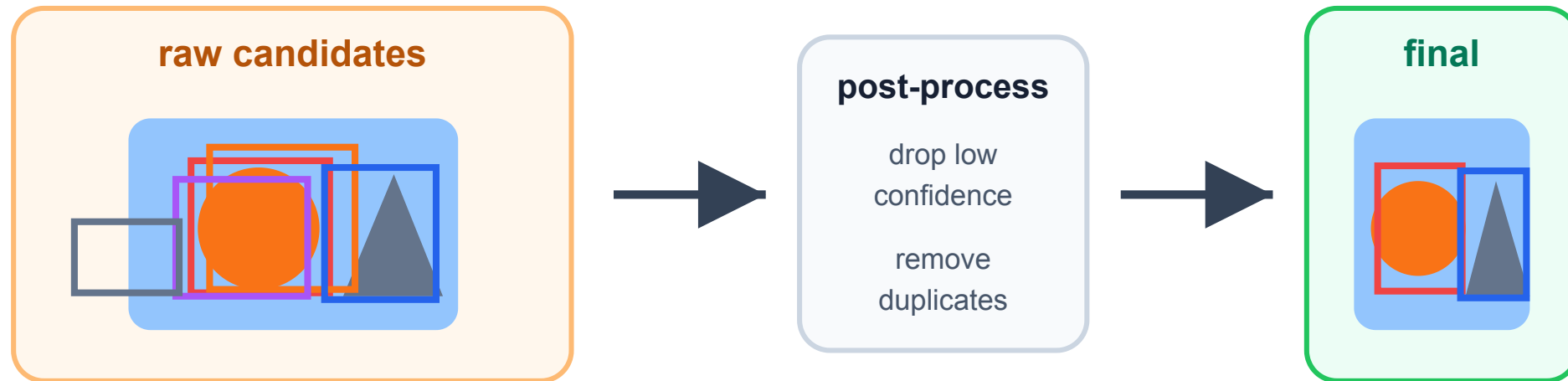
YOLO Pipeline

YOLO inference: predict candidates, then filter



Candidates vs Final Detections

Raw YOLO output is not the same as final visible detections



YOLO Architecture: Big Picture

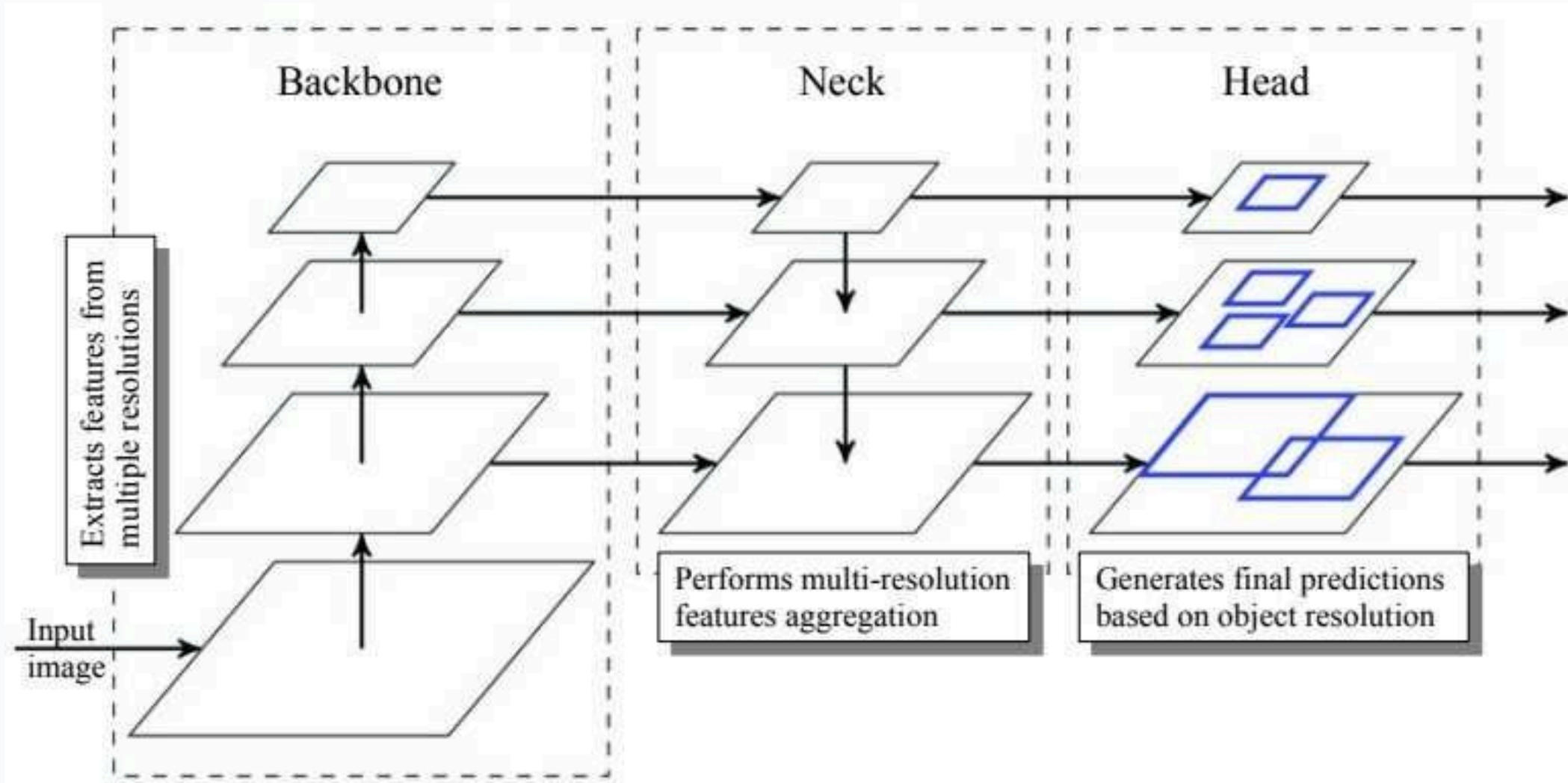
At a high level, YOLO-like detectors have three roles:

Part	Job
Backbone	extracts visual features
Neck	combines features at different scales
Detection head	predicts boxes, classes, and confidence

The exact layers change between YOLO versions.

The mental model stays the same.

YOLO Architecture: Picture



YOLO Architecture: What The Parts Do

YOLO-like detectors usually have three main parts.

Backbone:

extract visual features from the image

Neck:

combine feature maps from several resolutions

Detection head:

predict boxes, objectness/confidence, and class scores

The exact architecture changes across YOLO versions, but this decomposition is stable.

Why Multi-Scale Features?

Objects can be different sizes:

- small traffic signs
- medium people
- large cars

Deep features contain more semantic information.

Earlier features preserve more spatial detail.

Detection models combine scales to handle both small and large objects.

Detection Head Output

The detection head predicts many candidates.

Each candidate includes information like:

```
box coordinates  
object confidence  
class scores
```

Post-processing turns these candidates into the final visible boxes.

In practice, libraries expose these values through a result object.

What YOLO Predicts During Training

The network does not predict one final box.

It predicts many candidate detections across spatial locations and feature scales.

Each candidate contains:

```
box coordinates  
objectness / confidence  
class scores
```

For each candidate, the model learns to answer:

```
is there an object here?  
where is the box?  
what class is it?
```

What Is The Target?

Training starts from annotations:

```
class label + ground-truth bounding box
```

YOLO converts these annotations into detector targets:

```
responsible location / candidate  
target box coordinates  
objectness = 1 for matched object  
objectness = 0 for background candidates  
class target for the object
```

The model is trained to make its dense candidate predictions match these targets.

YOLO Loss: Three Jobs

YOLO-style losses combine several objectives.

Box loss:

predicted box should overlap the true box

Objectness loss:

object candidates high, background candidates low

Classification loss:

correct class for object candidates

Modern YOLO versions differ in exact formulas, but the training logic is this combination.

Ultralytics YOLO In Practice

Example usage:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model(image)
```

The goal is not to train YOLO.

The goal is to understand and analyze pretrained detection outputs.

Result Object

In a typical inference result, inspect:

```
boxes = results[0].boxes  
xyxy = boxes.xyxy  
cls = boxes.cls  
conf = boxes.conf
```

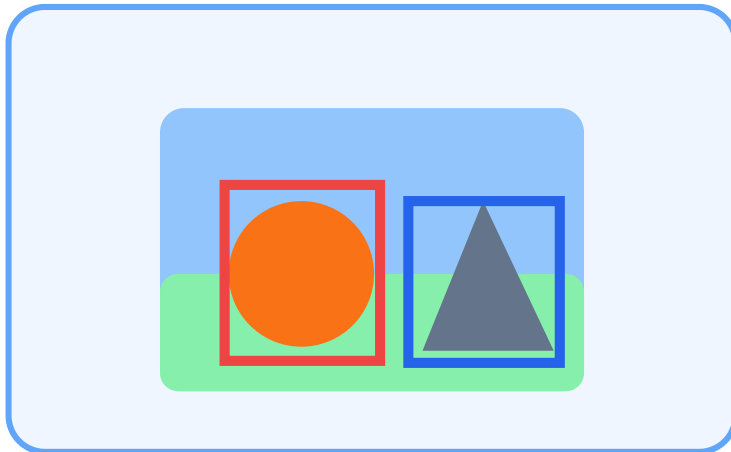
This gives:

```
coordinates  
class IDs  
confidence scores
```

Then you can render boxes on the image.

YOLO Result Object: Picture

Result tensors become rendered detections



xyxy	class	conf
[107,88,184,175]	dog	0.91
[198,96,273,176]	person	0.86
boxes.xyxy, boxes.cls, boxes.conf		

Precision And Recall

Precision asks:

of predicted detections, how many were correct?

Recall asks:

of true objects, how many did we find?

Confidence threshold affects both.

Usually:

higher threshold -> higher precision, lower recall
lower threshold -> lower precision, higher recall

Average Precision

As confidence threshold changes, precision and recall change.

AP summarizes the precision-recall curve:

AP = area under precision-recall curve

mAP averages AP:

mAP = mean AP over classes

For this course, understand the intuition, not the full benchmark protocol.

Evaluation Needs Matching

To evaluate detection, we need to decide whether a prediction matches a true object.

Usually a prediction is correct if:

```
class is correct  
IoU with ground-truth box is high enough
```

Example:

```
IoU  $\geq$  0.5
```

Different benchmarks may use different thresholds.

Part 5

Evaluation and failure analysis

Common Failure Modes

Detectors can fail by:

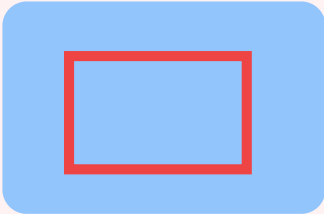
- missing small objects
- confusing similar classes
- detecting background as an object
- producing duplicate boxes
- failing under unusual lighting or viewpoint
- struggling with occlusion or crowding

These failure categories are useful when auditing detector behavior.

Failure Cases

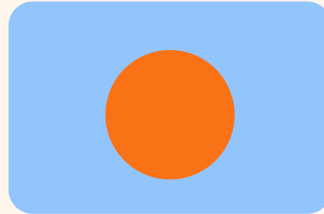
Detection quality needs visual inspection

false positive



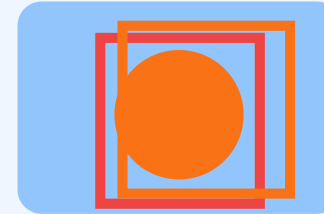
background detected

false negative



object missed

duplicates



NMS did not remove

Practical Threshold Advice

Start with default thresholds.

Then adjust based on application:

```
missing objects is costly -> lower confidence threshold  
false alarms are costly  -> higher confidence threshold  
duplicate boxes remain   -> adjust NMS IoU threshold
```

There is no universally best threshold.

It depends on the task.

What We Are Not Doing

Today we are not covering:

- YOLO loss derivation
- anchor design details
- version-by-version YOLO history
- implementing a detector from scratch

We focus on:

how to use, inspect, and reason about detector outputs
training objective intuition, not full implementation

Debugging Checklist

When using YOLO, check:

- input image loaded correctly
- detections render on the right image
- box format is understood
- class IDs map to names
- confidence threshold is appropriate
- NMS/IoU threshold is not hiding useful results
- failure cases are documented

Takeaways

Object detection predicts both:

what is present
where it is located

Key ideas:

- boxes describe object location
- IoU measures box overlap
- confidence thresholds trade precision and recall
- NMS removes duplicate boxes

Next lecture:

tokenization and transformer pretraining